

exp-mlkem-f203-stage12-encode-matmul-mix

F203 Stage1+2 融合方案 (encode + int8 MatMul)

ascendc 工程 (预研)

2026-05-19

摘要

本文档为 `examples/incubating/exp-mlkem-f203-stage12-encode-matmul-mix/` 的计划书：在 1 个 AI Core (`blockDim=1`, `LAUNCH_PROFILE=aicore=1`) 上融合 F203 Stage1 (`se [8,256] int32 → mat_a [16,256] int8`) 与 Stage2 (`mat_a × mat_b_lut [256,512] int8 → mat_c [16,512] int32`)。不含 Stage3 (RouteA + mod q)。

前置验证：Stage1 隔离 (`aiv=1/2/8`)、Stage2 隔离 (`aicore=1`) 均已 CPU 对拍通过；本用例为全链 Stage1→2 融合的第一步。核类型：`KERNEL_TYPE_MIX_AIC_1_2` (1 Cube + 2 Vector / AI Core)；同步参考 `exp-mlkem-sepolyvec8-ntt-k4` 的 `CrossCore*` 用法。

目录

1	背景与范围	3
1.1	为何做 Stage1+2 融合	3
1.2	本阶段不做	3
1.3	对齐的隔离用例	3
2	数学语义	3
2.1	输入输出	3
2.2	Stage1 (与 Stage1 customspec 一致)	3
2.3	Stage2 (与 Stage2 隔离一致)	4
2.4	Golden (黑盒 I/O)	4
3	AscendC 实现规划	4
3.1	AI Core 规模与验证档位	4
3.2	GM / Workspace 布局	4
3.3	融合流水 (单 AI Core)	5
3.4	Stage2 Tiling (<code>aicore=1</code> , 与隔离 Stage2 一致)	5
3.5	Stage1 分块 (建议)	5
4	全量 API 清单	5
4.1	Stage1 向量 (AIV)	5
4.2	Stage2 Cube (AIC) 与融合胶水	6

目录	2
5 逐步覆盖率评估	6
6 实现与验证（计划，本文档阶段不写代码）	7
6.1 计划文件（待用户放行后由 pre-research 创建）	7
6.2 Golden 与对拍	7
6.3 验收标准	7
7 风险与未决项	7

1 背景与范围

1.1 为何做 Stage1+2 融合

- 全链 F203 (sepolyvec8_ntt_f203) 在单核上须 Cube 与 Vector 协作; 隔离 Stage 已分别验证算术与 tiling。
- 本目录先融合 **Stage1+2**, 输出 mat_c 与 Stage2 隔离 golden 一致; 后续再与 Stage3 融合 (或接入既有全融合核)。

1.2 本阶段不做

- Stage3 RouteA、mod q 、输出 out [8,256]。
- 多 AI Core launch (aicore=4 等); 首版仅 aicore=1。
- Tag3 左 LUT 布局 [512,256] (本仓对齐 Alg13 行 16+17: LUT 在右 [256,512])。

1.3 对齐的隔离用例

用例	可复用语义
exp-mlkem-f203-stage1-encode-vec	Stage1 向量 encode; Sub 求 lo ; Cast 链; aiv=1 单核 8 poly 串行
exp-mlkem-f203-stage2-int8-matmul-cube	Stage2 Cube Matmul; aicore=1 tiling: SetDim(2)、SetSingleShape(16,256,K)、SetFixSplit(16,32,-1)
exp-mlkem-sepolyvec8-ntt-k4	MIX 骨架: KERNEL_TYPE_MIX_AIC_1_2; userWorkspace 中 mat_a/mat_c; CrossCoreSetFlag/CrossCoreWaitFlag

2 数学语义

2.1 输入输出

- 输入 se_polyvec_gm: [8,256] int32, 系数 $\in [0, q)$, $q = 3329$ 。
- 输入 mat_b_lut_gm: [256,512] int8 (由交付 input1 fp16 LUT 饱和; 与 Stage2 隔离一致)。
- 输出 mat_c_gm: [16,512] int32。
- 中间 mat_a: [16,256] int8, 布局 $[HI(8), LO(8)]$; 存于 userWorkspace GM (不写回 Host)。

2.2 Stage1 (与 Stage1 customspec 一致)

对每条 poly、系数 v :

$$hi = v \gg 6, \quad lo = v - hi \cdot 64 \quad (1)$$

写入 `mat_a` 行 p (HI) 与 $8+p$ (LO)。

2.3 Stage2 (与 Stage2 隔离一致)

$$C = A \times B, \quad A = \text{mat_a}, B = \text{mat_b_lut}, C = \text{mat_c} \quad (2)$$

A 为 16×256 int8, B 为 256×512 int8, C 为 16×512 int32 (ND, 无转置)。

2.4 Golden (黑盒 I/O)

`mlkem_ref.encode_mat_a(se) → matmul_int8_i32(mat_a, mat_b) → mat_c`。与 Stage2 `scripts/gen_data.py` 在相同 `se/LUT` 前提下输出一致 (Stage2 当前从交付 bin 读 `se` 再 encode, 融合用例应统一为 `mlkem_ref` 固定种子或同一交付 bin)。

3 AscendC 实现规划

3.1 AI Core 规模与验证档位

项	本用例约定
计划规模	仅 1 个 AI Core: <code>blockDim=1, LAUNCH_PROFILE=aicore=1</code>
命名	<code>run.sh --aicore 1</code> (两参数, 不用 <code>--aicore=1</code>); 环境变量 <code>LAUNCH_PROFILE=aicore=1</code>
核类型	<code>KERNEL_TYPE_MIX_AIC_1_2</code> : 每 AI Core 内 1×Cube (AIC) + 2×Vector (AIV)
SetDim	<code>usedCoreNum=2</code> : 参与 Matmul 流水的 Vector 个数 , 不是 2 个 AI Core (见 <code>qa/2026-06-09</code>)
Stage1 执行核	<code>ASCEND_IS_AIV</code> ; <code>GetBlockIdx()==0</code> 且 <code>GetSubBlockIdx()==0</code> 的单一 Vector 路径执行 encode (与全融合核一致)
Stage2 执行核	<code>ASCEND_IS_AIC</code> : <code>Matmul::IterateAll</code>
多档验证	首版仅 aicore=1 ; 通过后文档可扩展 aicore=4 (须另开 spec 修订, 不得实现擅自加档)
验证命令	<code>bash run.sh -r cpu -v Ascend910B4 --aicore 1 → mat_c_gm.bin</code> 与 <code>golden.bin</code> 一致

选型理由: 用户指定首版单核融合; 与 Stage2 已验证的 `aicore=1` tiling 闭合; 为后续 Stage3 接入全融合核提供最小 MIX 路径。

3.2 GM / Workspace 布局

- `userWorkspaceGm` 连续区 (Host `aclrtMalloc` / CPU `GmAlloc`):
 - 偏移 0: `mat_a`, 16×256 int8 (4096 B)

– 偏移 4096: `mat_c`, 16×512 int32 (32768 B)

- `tilingGm`: TCubeTiling + UB 大小(同 `Stage2 mlkem_stage2_matmul_custom_tiling.cpp`)。
- `workspaceGm`: Matmul 库 workspace (`GetSysWorkSpacePtr / REGIST_MATMUL_OBJ`)。

3.3 融合流水 (单 AI Core)

1. **Stage1 (AIV)**: 读 `se_gm` $\rightarrow$ 写 `mat_a` (workspace); 8 poly 串行, 每 poly 可新建向量 op (对齐 Stage1 aiv=1, 避免 TPipe 跨 poly 复用)。
2. **同步**: Stage1 末 `CrossCoreSetFlag<2, PIPE_MTE3>(syncId)(AIV)`; AIC 侧 `CrossCoreWaitFlag(syncId)` 后再读 `mat_a`。
3. **Stage2 (AIC)**: Matmul int8 $\times$ int8 $\rightarrow$ int32, 写 `mat_c` (workspace 偏移 +4096)。
4. **Host**: 仅 D2H 拷贝 `mat_c` 至 `output/mat_c_gm.bin`。

3.4 Stage2 Tiling (aicore=1, 与隔离 Stage2 一致)

- $M = 16$, $N = 512$, $K = 256$; `SetOrgShape/SetShape(16,512,256)`。
- `SetSingleShape(16,256,K)`; `SetFixSplit(16,32,-1)`; `SetDim(2)`。
- int8 入 int32 出最小 tile $16 \times 32 \times 16$ 粒度须满足 (见全融合 实现方案.tex §Tiling)。

3.5 Stage1 分块 (建议)

首版可采用与 `exp-mlkem-f203-stage1-encode-vec` 相同 `kTileLength=32(kTileNum=4,kBufferNum=2)`, 或对齐全融合核 `kStage1TileLen=64`; 实现时二选一并在代码头注释写明, 对拍不变。

4 全量 API 清单

说明: 长表用 `L{}` + `\apiname`; 每个 API 单独一行。

4.1 Stage1 向量 (AIV)

API	分类	在线文档 ID	A2	本用例 dtype	备注
<code>DataCopy</code>	2.3.1	07_0101	✓	int32/int8	读 <code>se</code> 、写 <code>mat_a</code>
<code>ShiftRight</code>	2.3.3	07_0059	✓	int32	$hi = v \gg 6$
<code>Muls</code>	2.3.3	07_0055	✓	int32	$hi \times 64$
<code>Sub</code>	2.3.3	07_0036	✓	int32	$lo = v - hi \cdot 64$
<code>Cast</code>	2.3.3	07_0073	✓	int32 $\rightarrow$ int8	链: i32 $\rightarrow$ i16 $\rightarrow$ half $\rightarrow$ i8
<code>TPipe</code>	2.3 资源	07_0108	✓	—	向量流水
<code>InitBuffer</code>	2.3 资源	07_0110	✓	—	TQue/TBuf

API	分类	在线文档 ID	A2	本用例 dtype	备注
TQue	2.3 Pipe/Que	07_0137	✓	—	双缓冲
TBuf	2.3 Pipe/Que	07_0137	✓	—	VECCALC

4.2 Stage2 Cube (AIC) 与融合胶水

API	分类	在线文档 ID	A2	本用例 dtype	备注
Matmul	2.3.2 / 库	Matmul 教程	✓	int8×int8→int32	REGIST_MATMUL_OBJ
REGIST_MATMUL_OBJ	Cube 库	样例	✓	—	绑定 TPipe
MultiCoreMatmulTiling	Tiling API	CANN tiling	✓	—	Host
CrossCoreSetFlag	Utils / 融合	API 列表 ≈p.15	✓	事件 ID	GenerateTiling Stage1 完成 → AIC
CrossCoreWaitFlag	Utils / 融合	API 列表 ≈p.15	✓	事件 ID	AIC 等待 mat_a
GetSubBlockIdx	系统变量	07_0185 附近	✓	—	区分 AIV0/1
ASCEND_IS_AIC	宏	编程指南	✓	—	核角色分支
ASCEND_IS_AIC_V	宏	编程指南	✓	—	核角色分支
KERNEL_TASK_TYPE_DEFAULT	Utils	编程指南	✓	MIX_AIC_1_2	融合核

5 逐步覆盖率评估

步	阶段	API / 机制	覆盖	备注
F0	launch	MIX_AIC_1_2, blockDim=1	100%	单 AI Core
S1	encode	Stage1 向量 API 表	100%	隔离 Stage1 已验
X1	S1→S2	CrossCore*	100%	须保证 mat_a 对 AIC 可见
S2	matmul	Matmul + tiling	100%	隔离 Stage2 aicore=1 已验

H1	写回 Host	DataCopy (main)	100%	仅 mat_c
主路径 (Stage1+2)			100%	无 Stage3 标量缺口

6 实现与验证（计划，本文档阶段不写代码）

6.1 计划文件（待用户放行后由 pre-research 创建）

- f203_stage12_encode_matmul_mix_custom.cpp —MIX 核
- f203_stage12_encode_matmul_mix_custom_tiling.cpp —tiling (aicore=1)
- launch_profile.h、main.cpp、run.sh、scripts/gen_data.py、scripts/verify_result.py

6.2 Golden 与对拍

- input/se_polyvec_gm.bin、input/mat_b_lut_gm.bin
- output/golden.bin: [16,512] int32
- output/mat_c_gm.bin 与 golden cmp + verify_result.py

6.3 验收标准

- bash run.sh -r cpu -v Ascend910B4 --aicore 1 → max_abs_diff=0
- 输出与 exp-mlkem-f203-stage2-int8-matmul-cube 在相同输入下 mat_c 一致（间接验证 Stage1 写入 workspace 正确）

7 风险与未决项

- **TPipe 生命周期**: MIX 核内 AIV/AIC 是否共享同一 TPipe 实例须对照 sepolyvec8_ntt_custom.cpp; Stage1 跨 poly 仍建议每 poly 新建向量类。
- **同步事件 ID**: CrossCore 的 syncId 勿与库内保留值冲突（全融合用 7; 本用例可复用或单独分配，实现时注释）。
- **Stage1 tile 宽度**: 64 vs 32 仅影响性能，实现前在 STATUS 固定一种。